

Chamada de Procedimento Remoto

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Motivação para RPC

Dificuldades na comunicação entre processos via sockets

Cliente em Java

- 1 Conecta no servidor
- 2 Lê do teclado dois long
- 3 Converte long para String
- 4 Envia as Strings
- 5 Recebe o resultado

Servidor em C

- 1 Recebe via sockets vetor de char
- 2 Converte char para long
- 3 Faz a soma dos dois long
- 4 Devolve resultado como vetor de char

Serialização

Traduz estruturas de dados para sequência de bytes de forma que possa ser armazenado em disco ou transportado pela rede

Motivação para RPC

Dificuldades na comunicação entre processos via sockets

- **Comunicação** entre processos clientes e servidores por meio de **troca explícita de mensagens**
 - Ex: Aplicação Java enviando objetos do tipo Mensagem

Mensagem: msg
+ cod : int = 100
+ corpo : String = "Jogador 1"

```
// send
saida.writeObject(msg);
//receive
Mensagem resp = entrada.readObject();
```

Motivação para RPC

Dificuldades na comunicação entre processos via sockets

- **Comunicação** entre processos clientes e servidores por meio de **troca explícita de mensagens**
 - Ex: Aplicação Java enviando objetos do tipo Mensagem

Mensagem: msg

```
+ cod : int = 100  
+ corpo : String = "Jogador 1"
```

```
// send  
saida.writeObject(msg);  
//receive  
Mensagem resp = entrada.readObject();
```

Não provê a **transparência de acesso**

- *Esconder as diferenças para representação dos dados*
- *Como os dados serão representados em diferentes arquiteturas*

Inspiração para RPC: Chamada de procedimentos em C

```
#include <stdio.h>
void funcao_a(int *r){
    (*r)++;
    .....
}

int main(void){
    int v = 1;
    .....
    funcao_a(&v);
    .....
    return 0;
}
```

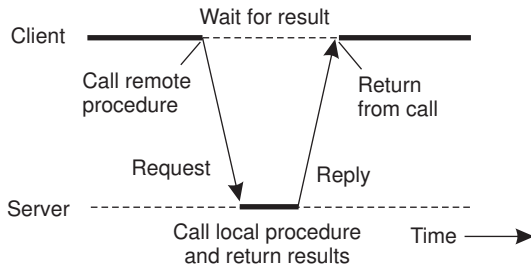
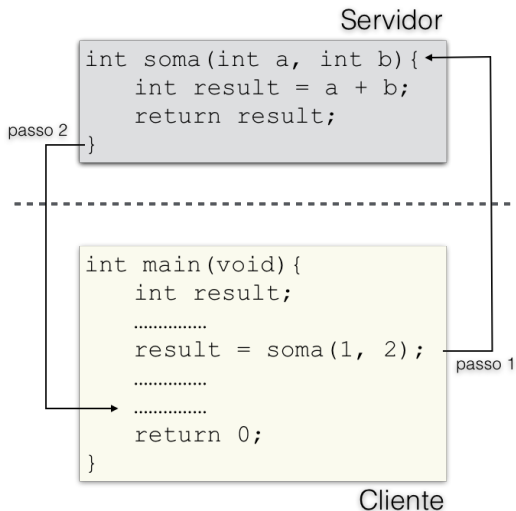
■ Transferência de controle e dados

- Lista de parâmetros e tipos de retorno permitem a comunicação entre funções

■ Linguagem C: passagem de parâmetros pode ser por

- Valor
 - Função que fora invocada cria uma cópia local da variável
- Referência
 - Função que fora invocada recebe endereço de memória da variável

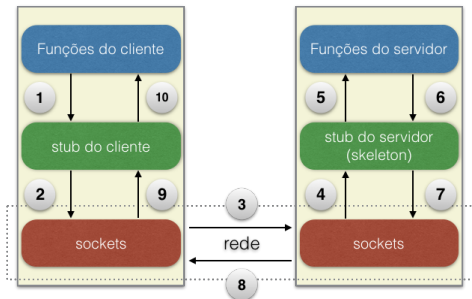
Chamada de procedimento remoto – RPC



■ Chamada síncrona

Chamada de procedimento remoto – RPC

- Processos locais realizam chamadas de **procedimentos** que estão **localizados em outras máquinas**



- 1 Cliente invoca funções do stub
- 2 Stub serializa dados
- 3 Dados são transmitidos via sockets
- 4 Dados recebidos são entregues para o stub
- 5 Dados são deserializados
- 6 Resposta segue caminho inverso

Stub do cliente e Skeleton do servidor

■ *stub* do cliente

- Atua como um *proxy* para as funções remotas do servidor
- Serialização dos parâmetros e envio de mensagens pela rede
- Aguarda pela resposta do servidor e faz a deserialização

■ *skeleton* do servidor

- Aguarda pelos pedidos dos clientes e ao receber, invoca a função do servidor
- Faz a deserialização para encaminhar ao servidor
- Serializa a resposta e envia ao cliente



Serialização é o processo de transformar uma estrutura de dados em um formato que possa ser armazenado ou transmitido pela rede, ou seja, em um fluxo de *bytes*.

Representação dos dados

Marshalling

Semelhante a serialização, porém consiste na transformação da **representação** de um objeto em memória em um formato que possa ser armazenado ou transportado pela rede

- Formato de representação com **tipo de variável implícito**
 - Somente os valores são transmitidos
 - Exemplo: XDR (*eXternal Data Representation*)
- Formato de representação com **tipo de variável explícito**
 - Além do valor, o tipo da variável também é transmitido
 - Exemplos: ASN.1, XML, JSON e Google Protocol Buffers

Implementação ONC RPC

- Em 1980 a **SUN RPC** foi a primeira implementação
 - Usada no SUN Network File System (NFS)
- Padronizada pela Open Network Computing (**ONC**)
 - Presente no SunOS, BSDs, Linux e macOS
- Microsoft faz uso do seu próprio RPC (DCOM + Object RPC)
 - Surgiu em 1996 com o Windows NT 4.0

Implementação ONC RPC

- Em 1980 a **SUN RPC** foi a primeira implementação
 - Usada no SUN Network File System (NFS)
- Padronizada pela Open Network Computing (**ONC**)
 - Presente no SunOS, BSDs, Linux e macOS
- Microsoft faz uso do seu próprio RPC (DCOM + Object RPC)
 - Surgiu em 1996 com o Windows NT 4.0



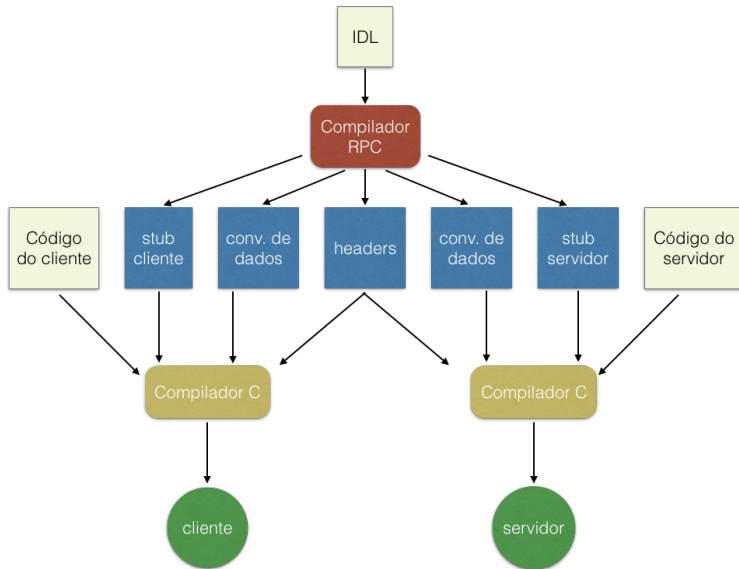
O gRPC é um *framework* moderno que permite a comunicação transparente entre aplicações clientes e servidoras. Faz uso do *Protocol Buffers* como IDL e para representação das mensagens.

Linguagem de Descrição de Interfaces

Interface Description Language (IDL)

- Linguagem para descrever interface de componentes de software
- Descrição de interface independente de qualquer linguagem de programação e arquitetura de máquina
- Possibilita a comunicação entre componentes escritos em diferentes linguagens
- **RPC faz uso de IDL** para descrever os procedimentos remotos expostos por um servidor

Passos para escrever cliente e servidor com ONC RPC



Como escrever programas com ONC RPC

- Descrever a interface dos procedimentos (IDL RPC)

```
/* Dois procedimentos que recebem uma struct,  
   faz a computação e retorna o resultado (int) */  
struct operandos{  
    int a; int b;  
};  
  
program CALCULADORA_PROG{  
    version CALCULADORA_VERSION{  
        int SOMA(operandos) = 1;  
        int SUB(operandos) = 2;  
    } = 1; /* versão do procedimento */  
} = 0x20000001; /* versão do programa */
```

- Compilar IDL para gerar stubs para cliente e servidor

```
rpcgen calculadora.x # ou rpcgen -a -C calculadora.x
```

Servidor

```
#include "calculadora.h"
int * soma_1_svc(operandos *argp, struct svc_req *rqstp){
    static int result;
    result = argp->a + argp->b;
    return &result;
}
```

Cliente

```
#include "calculadora.h"
int main(void){
    char *host = "127.0.0.1";
    CLIENT *clnt;
    operandos argumentos;
    int *resultado;

    clnt = clnt_create(host, CALCULADORA_PROG, CALCULADORA_VERSION, "udp");
    if (clnt == NULL) { clnt_pcreateerror(host); exit(1); }

    argumentos.a = 1; argumentos.b = 2;
    resultado = soma_1(&argumentos,clnt); /* invocando procedimento remoto */
    printf("Resultado da soma: %d", *resultado);
}
```


Compilando e executando

```
# Servidor
```

```
gcc servidor.c calculadora_svc.c calculadora_xdr.c -lnsl -o servidor
```

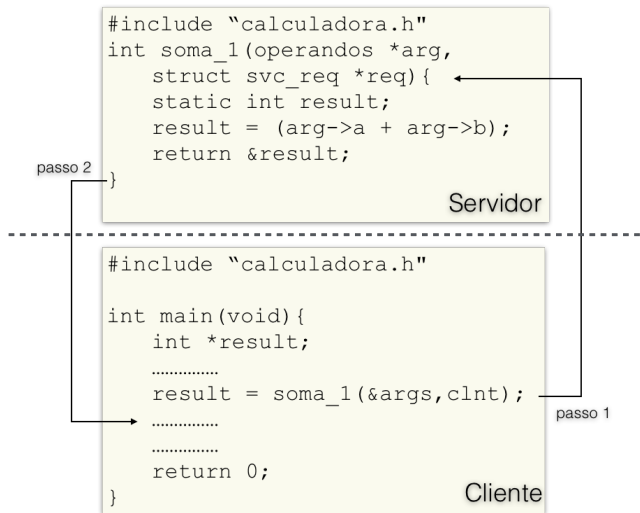
```
./servidor
```

```
# Cliente
```

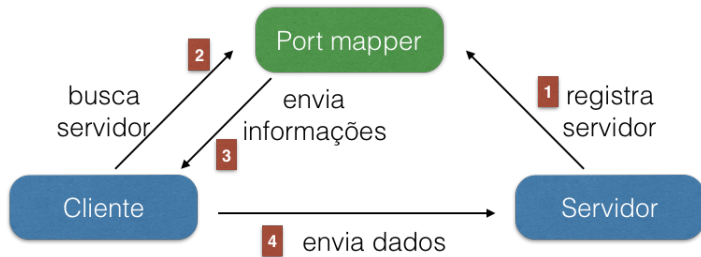
```
gcc cliente.c calculadora_clnt.c calculadora_xdr.c -lnsl -o cliente
```

```
./cliente
```

Chamada de procedimento remoto – RPC



Em qual porta o servidor está ouvindo?



- **Port mapper** é um serviço de descoberta que permite aos clientes descobrirem em qual porta TCP/UDP o servidor está ouvindo
 - Executa por padrão na porta 111 (TCP ou UDP)
 - Versões 3 e 4 são chamadas de `rpcbind` protocol
- Faça o teste no teu S.O. e veja quais são os serviços registrados

```
/usr/sbin/rpcinfo -p
```

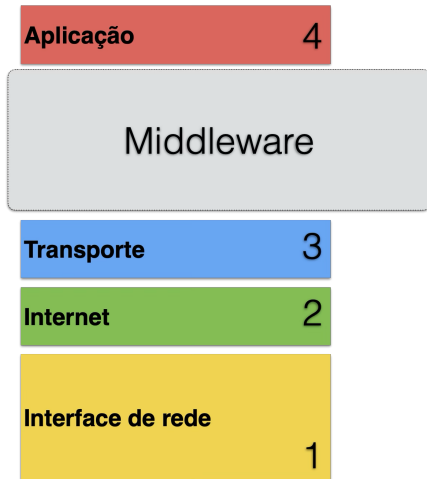
Resumo sobre RPC



modelo OSI

- **Camada 6** – Representação e serialização dos dados
- **Camada 5** – Gerenciamento de conexão

Resumo sobre RPC



Arquitetura TCP/IP

- **Camada 6** – Representação e serialização dos dados
- **Camada 5** – Gerenciamento de conexão

Resumo sobre RPC

■ Protocolo SOAP

- Protocolo e representação dos dados em XML
- Empregado na construção de *Web Services*

■ gRPC

- Protocolo e representação dos dados com *Protocol Buffers*
- Faz uso do protocolo HTTP/2 como transporte
- Empregado na arquitetura de microserviços



Curiosidade

Em <http://www.http2demo.io> tem uma demonstração comparando HTTP/1.1 com HTTP/2

Remote Method Invocation (Java RMI)

Objetos distribuídos e invocação de métodos remotos

■ Chamada Remota de Procedimento – RPC

- Permite que processos locais realizem chamadas de procedimentos que estão localizados em outras máquinas

Objetos distribuídos e invocação de métodos remotos

■ Chamada Remota de Procedimento – RPC

- Permite que processos locais realizem chamadas de procedimentos que estão localizados em outras máquinas

■ Invocação de Métodos Remotos – Java RMI

- **Semelhante ao RPC**, porém voltado para **objetos distribuídos**
- Estende o conceito de referência de objetos para um ambiente global distribuído
 - Objeto residente em um processo pode invocar métodos de um objeto residente em um outro processo
- Oferece *distributed garbage-collection*

RPC vs RMI

- Semelhanças entre RPC e RMI

- Diferenças entre RCP e RMI

RPC vs RMI

■ Semelhanças entre RPC e RMI

- Fazem uso de linguagem de descrição de interfaces (IDL)
- São construídos sobre protocolos pedido e resposta
- Oferecem o mesmo nível de transparência ao desenvolvedor

■ Diferenças entre RCP e RMI

RPC vs RMI

■ Semelhanças entre RPC e RMI

- Fazem uso de linguagem de descrição de interfaces (IDL)
- São construídos sobre protocolos pedido e resposta
- Oferecem o mesmo nível de transparência ao desenvolvedor

■ Diferenças entre RCP e RMI

- Com o RMI o desenvolvedor poderá usufruir das facilidades da POO
 - objetos, classes, herança, ...
- Objetos possuem uma referência única em todo o sistema
 - Objetos podem ser passados por referência (e não somente por valor)

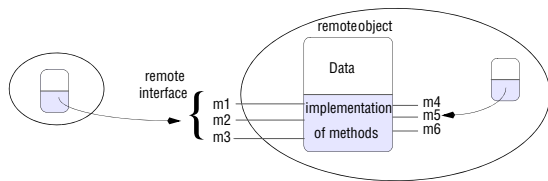
Modelo de objetos distribuídos

■ Referência de objeto remoto

- um objeto só poderá invocar métodos de outros objetos que este possuir a referência

■ Interface remota

- cada objeto remoto possui uma interface que especifica quais de seus métodos poderão ser invocados remotamente
- objetos dentro de um mesmo processo poderão invocar os métodos remotos e também os locais



Entidades participantes

■ Servidor

- Processo que oferta objeto remoto

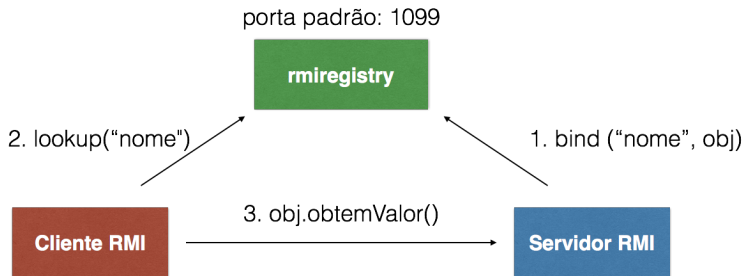
■ Cliente

- Processo que invoca um método de um objeto remoto

■ Serviço de registro

- Serviço de nomes que associa nomes a objetos
- Ex: `rmi://host:port/pathName`

Java RMI



- Serviço de registro pode ser iniciado na linha de comando:

```
rmiregistry 12345 &
```

- ou dentro do código Java do processo servidor:

```
int PORTA = 12345;  
Registry registro = LocateRegistry.createRegistry(PORTA);
```

Remote Method Invocation (Java RMI)

Exemplo: Um Contador

Descrição da interface

```
package std;  
  
public interface ContadorDistribuido extends Remote{  
    public void incrementa() throws RemoteException;  
    public int obterValor() throws RemoteException;  
}
```



Essa interface Java deve ser compartilhada entre servidor e clientes. Deve-se manter o mesmo nome de pacote Java.

Implementação da Interface no Servidor

```
package std;

public class Contador implements ContadorDistribuido{
    public int valor = 0;

    public void incrementa() throws RemoteException {
        this.valor++;
    }

    public int obtenValor() throws RemoteException {
        return this.valor;
    }
}
```

Implementação do Servidor

```
package std;

public class Servidor{

    public static void main(String[] args) throws Exception{
        Contador c = new Contador(); // criando objeto

        ContadorDistribuido stub =
            (ContadorDistribuido) UnicastRemoteObject.exportObject(c, 0);

        // Criando registry
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        Registry registro = LocateRegistry.createRegistry(12345);

        // Registrando objeto com o nome MeuContador
        registro.bind("MeuContador", stub);

        System.out.println("Servidor pronto!");
    }
}
```

Implementação do Cliente

```
package std;

public class Cliente{

    public static void main(String[] args) throws Exception{

        // Obtendo referência para o registro (tem que conhecer IP e porta)
        Registry registro = LocateRegistry.getRegistry("127.0.0.1", 12345);

        // Obtendo referência para o objeto instanciado pelo Servidor
        ContadorDistribuido stub =
            (ContadorDistribuido) registro.lookup("MeuContador");

        // invocando métodos do objeto remoto
        System.out.println("valor: " + stub.obtemValor());
        stub.incrementa();
        System.out.println("valor: " + stub.obtemValor());
    }
}
```

Aulas baseadas em



COULORIS, George; DOLLIMORE, Jean; KINDBERG, Tim. **Sistemas Distribuídos: Conceitos e projeto**. 5. ed.: Bookman, 2013.

<https://app.minhabiblioteca.com.br/books/9788582600542/>.



KRZYZANOWSKI, Paul. **Distributed Systems – Rutgers University**. 2020.



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.